Pointers

Pointers

- **<u>pointer</u>** – a variable whose value is a reference to some object
  - pointer use may be restricted or not
    - only allow pointers to point to heap (e.g. Pascal)
    - allow “address of” operator (e.g. ampersand in C)
  - pointers not equivalent to addresses!
  - how reclaim heap space?
    - explicit (programmer’s duty)
    - garbage collection (language implementation’s duty)

Value Model – More on C

- Pointers and single dimensional arrays interchangeable, though space allocation at declaration different

```
int a[10]; int *b;
```

- For subroutines, pointer to array is passed, not full array
- Pointer arithmetic
  - <Pointer, Integer> addition

```
int a[10];
int n;
n = *(a+3);
```

  - <Pointer, Pointer> subtraction and comparison

```
int a[10];
int * x = a + 4;
int * y = a + 7;
int closer_to_front = x < y;
```

Dangling References

- **<u>dangling reference</u>** – a live pointer that no longer points to a valid object
  - to heap object: in explicit reclamation, programmer reclaims an object to which a pointer still refers
  - to stack object: subroutine returns while some pointer in wider scope still refers to local object of subroutine
- How do we prevent them?

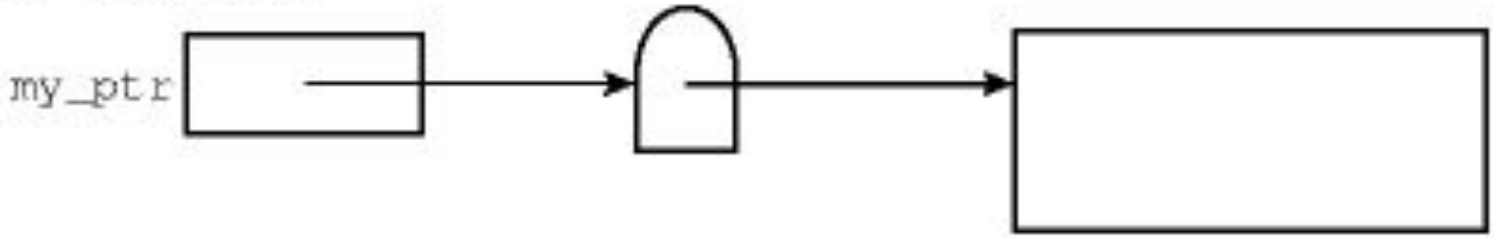
Dangling References

- Prevent pointer from pointing to objects with shorter lifetimes (e.g. Algol 68, Ada 95). Difficult to enforce
- Tombstones
- Locks and Keys

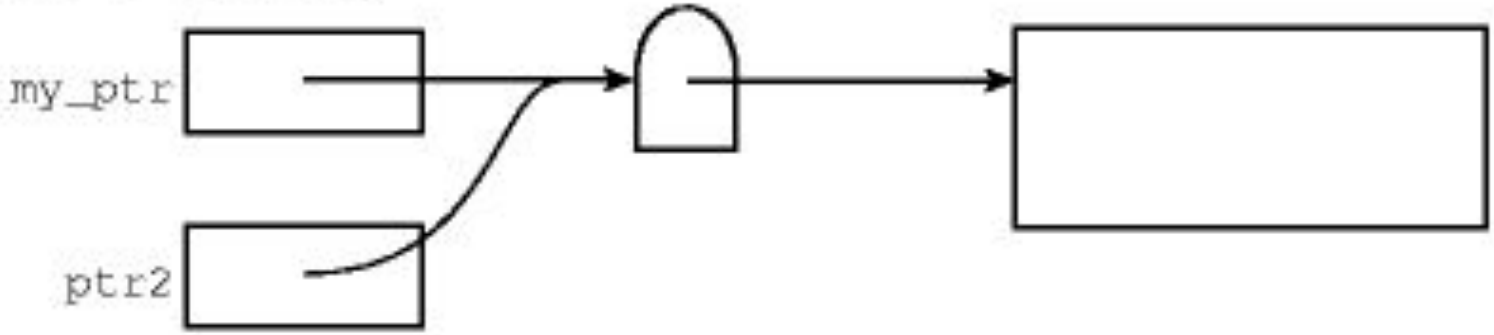
Tombstones

- Idea
  - Introduce another level of indirection: pointer contain the address of the tombstone; tombstone contains address of object
  - When object is reclaimed, mark tombstone (zeroed)
- Time overheads
  - Create tombstone
  - Check validity of access
  - Double indirection
- Space overheads
  - when to reclaim??
- Extra benefits
  - easy to compact heap
  - works for heap and stack

```
new (my_ptr);
```




```
ptr2 := my_ptr;
```

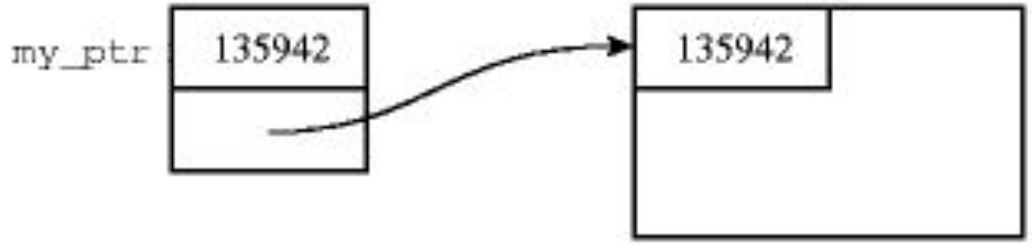



```
delete (my_ptr);
```

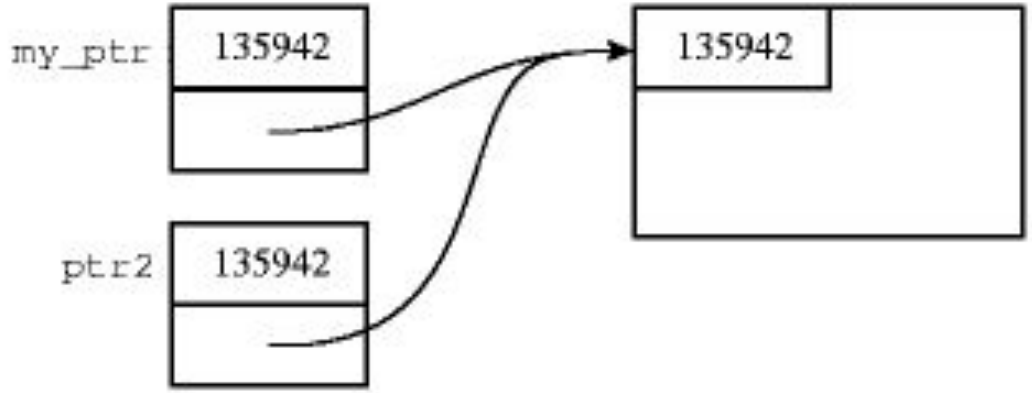
Locks and Keys

- Idea
  - Every pointer is `<address, key>` tuple
  - Every object starts with same lock as pointer's key
  - When object is reclaimed, object's lock marked (zeroed)
- Advantages
  - No need to keep tombstones around
- Disadvantages
  - Objects need special key field (usually implemented only for heap objects)
  - Probabilistic protection
- Time overheads
  - Lock to key comparison costly

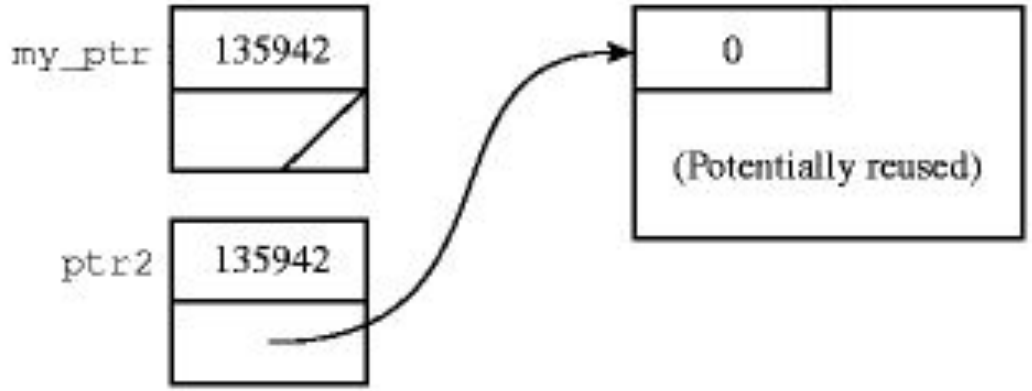
```
new (my_ptr);
```




```
ptr2 := my_ptr;
```




```
delete (my_ptr);
```



Garbage Collection

- Language implementation notices when objects are no longer useful and reclaims them automatically
  - essential for functional languages
  - trend for imperative languages
- When is object no longer useful?
  - Reference counts
  - Mark and sweep
  - “Conservative” collection